

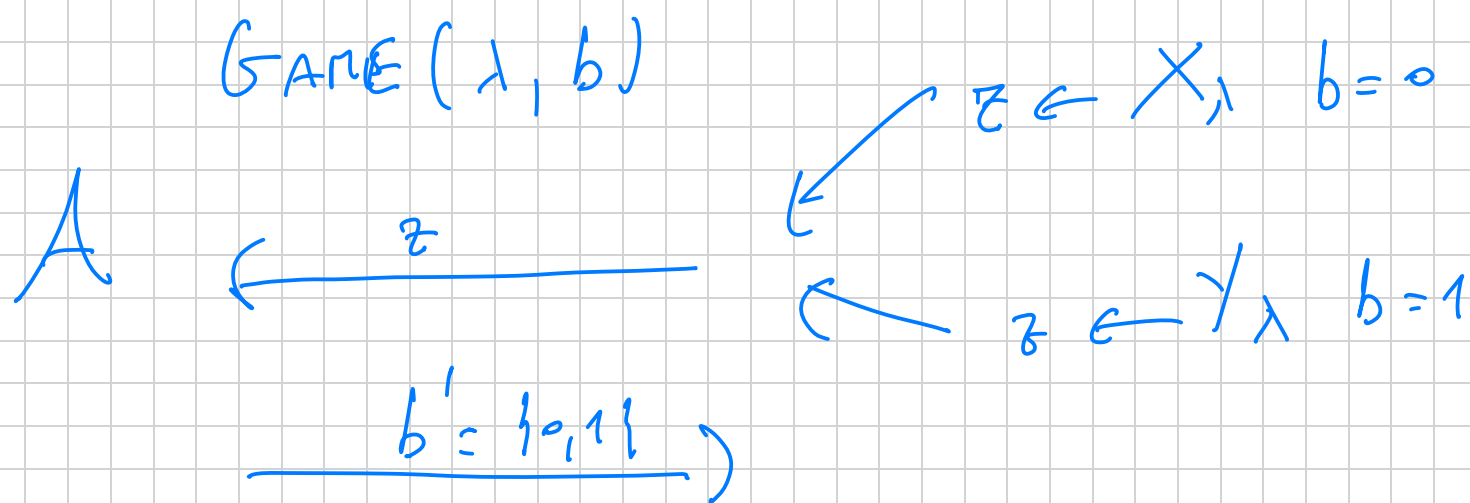
PSEUDORANDOMNESS

A deterministic function that takes a uniform string and outputs many bits that "look" random!

Pseudorandom: No PPT A can distinguish it from random except with negligible probability.

DEF Let $X = \{X_\lambda\}_{\lambda \in \mathbb{N}}$, $Y = \{Y_\lambda\}_{\lambda \in \mathbb{N}}$. We say that $X \approx_c Y$ if $\forall$ PPT A :

$$\left| \Pr[A(X_\lambda) = 1] - \Pr[A(Y_\lambda) = 1] \right|$$
$$\hookrightarrow \Pr[A(z) = 1; z \leftarrow X_\lambda] \leq \text{negl}(\lambda)$$



$$\{\text{GAME}(\lambda, 0)\} \approx_c \{\text{GAME}(\lambda, 1)\}$$

With this step now we can define PSEUDORANDOM GENERATORS.

DEF A function $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{\lambda+l}$ ($l > 0$)
 is a (secure) PRG if

$$\{X_\lambda\} = \{G(U_\lambda)\}_\lambda \approx_c \{U_{l+\lambda}\} = \{Y_\lambda\}$$

EXERCISE. No PRG is secure against unbounded attackers. Not even if $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{\lambda+1}$

Solution: Given $z \in \{0,1\}^{\lambda+1}$ try to find $s \in \{0,1\}^\lambda$ s.t. $z = G(s)$. If you succeed output 1, else 0.

$$\Pr[A(z) = 1 : z = G(s); s \leftarrow U_\lambda] = 1$$

$$\Pr[A(z) = 1 : z \leftarrow U_{\lambda+1}] \leq \frac{1}{2}$$

How to construct G ?

THM If OWFs exist so do PRGs with stretch $l(\lambda) = \text{poly}(\lambda)$.

The proof is hard, but we will discuss some important elements. Two ingredients:

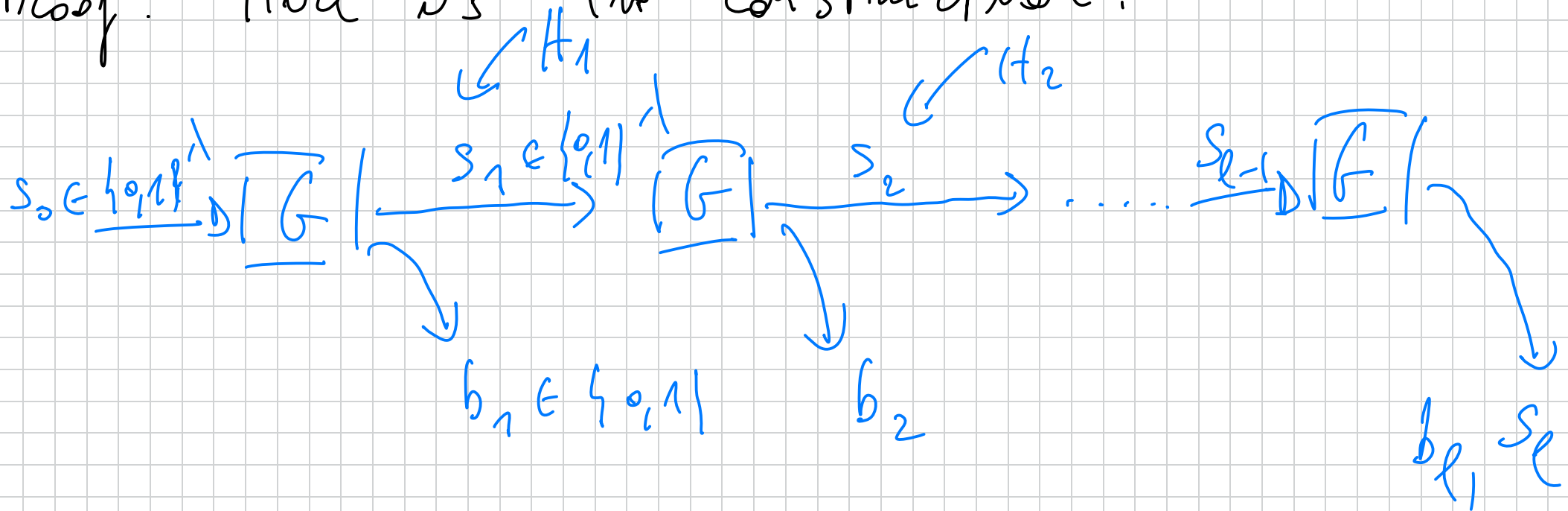
1) From $l(\lambda) = 1$ to $l(\lambda) = \text{poly}(\lambda)$,
or stretch amplification.

2) Hard-core bits.

Thm

Assume $G: \{0,1\}^l \rightarrow \{0,1\}^{l+1}$ is a PRG. Then there is a PRG $G^l: \{0,1\}^l \rightarrow \{0,1\}^{l+l}$ for every $l(1) = \text{poly}(l)$.

Proof. Here is the construction.



$$(s_i, b_i) = G(s_{i-1})$$

More formally:

- let $s_0 \in U_1$

- $\forall i \in [l], (s_i, b_i) = G(s_{i-1})$

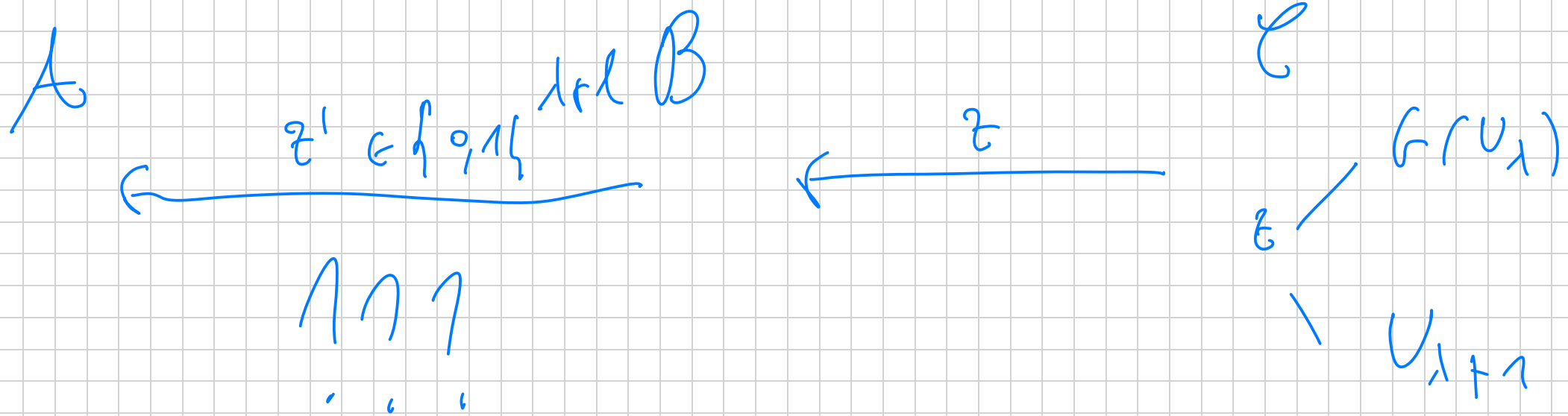
$s_i \in \{0,1\}^k, b_i \in \{0,1\}$

- Output: $(b_1, \dots, b_l, s_l)$

How to prove it? By reduction! Meaning:

assume $\exists$ PPT A breaking G^k , Then

we build a PPT B breaking G .



To solve this efficiently, we introduce a new technique: The HYBRID ARGUMENT.

If we need to show $X \approx_c Y$ we can do so by finding HYBRIDS $H_0, H_1, \dots, H_\ell$ s.t.

$$(i) \quad H_0 \equiv X ; \quad H_\ell = Y.$$

$$(ii) \quad H_0 \sim_c H_1, \quad H_1 \sim_c H_2 ; \dots ; H_{\ell-1} \sim_c H_\ell$$

By the triangle inequality, $H_0 \sim_c H_\ell$ so long as $\ell(\lambda) = \text{poly}(\lambda)$.

In our case:

$$H_0 \equiv G^{\ell}(U_{\lambda}) ; \quad H_\ell \equiv U_{\ell+1}$$

$$b_1, \dots, b_i \leftarrow \{0, 1\}$$

H_i :

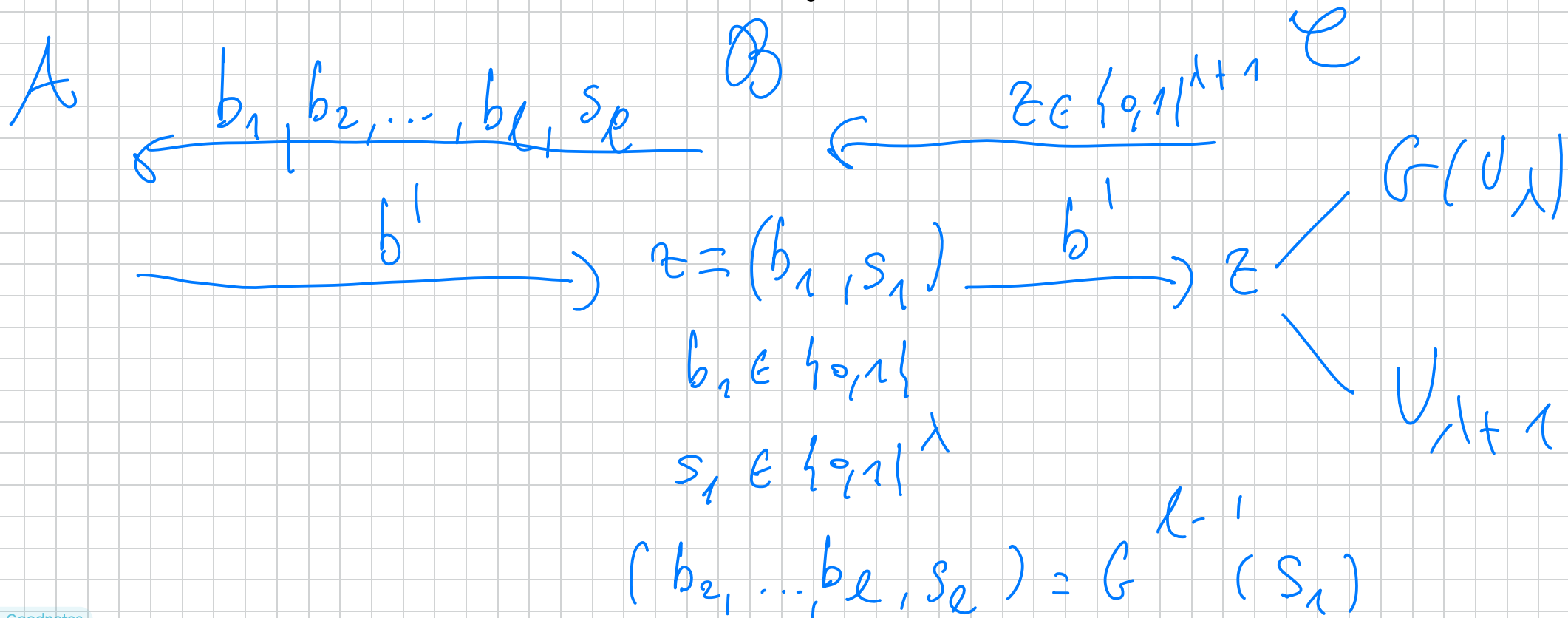
$$s_i \leftarrow U_{\lambda}$$

$$(b_{i+1}, \dots, b_\ell, s_\ell) = G^{\ell-i}(s_i)$$

Now, we can make the reduction work!

Let's be concrete: $H_0 \sim_c H_1$. Assume not:
 $\exists$ PPT A that distinguishes H_0 from H_1 .

We build B breaking G .



Observe: If $z \in G(U_\lambda)$ then $b_1, \dots, b_\ell, s_\ell$ computed by B are distributed like H_0

Else, if $z \in U_{\lambda+1}$ then it is distributed like H_1 .

By assumption:

$$\begin{aligned} & \left| \Pr[A(b_1, \dots, b_\ell, s_\ell) = z] - \Pr[A(b_1, \dots, b_\ell, s_\ell) = z] \right| \\ &= \left| \Pr[A(b_1, \dots, b_\ell, s_\ell) = z] - \Pr[A(b_1, \dots, b_\ell, s_\ell) = z] \right| \\ &\geq \frac{1}{p(\lambda)} \quad \text{for some polynomial } p(\lambda). \end{aligned}$$

And we have shown:

$$Pr [B(z) = 1 : z \leftarrow G(U_\lambda)] =$$

$$Pr [A(b_1, \dots, b_\ell, s_\ell) = 1 : b_1, \dots, b_\ell, s_\ell \leftarrow H_0]$$

$$Pr [B(z) = 1 : z \leftarrow U_{\lambda+1}] =$$

$$Pr [A(b_1, \dots, b_\ell, s_\ell) = 1 : b_1, \dots, b_\ell, s_\ell \leftarrow H_1]$$

$\Rightarrow$ So B distinguishes s w.p. $\frac{1}{p(\lambda)}$

By very standard proof:

$$\forall n' \quad H_n \approx_c H_{n+1}$$

QED

How do we get $G: \{0,1\}^l \rightarrow \{0,1\}^{l+1}$ in the first place? In theory: I can do it starting with ANY owf! Not very efficient. But we will see later efficient constructions using NUMBER THEORY, LATTICES, ...

The main ingredient is HARD-CORE BITS.

Let $f: \{0,1\}^M \rightarrow \{0,1\}^M$ be a owf. Can we say that a particular bit of x is hard to compute ???

Intuitively, $h: \{0,1\}^M \rightarrow \{0,1\}$ is HARD CORE if $h(x)$ is hard to compute given $f(x)$.

Alternatively : $(f(x), h(x)) \approx_c (f(x), u)$
 $u \leftarrow \{0, 1\}^m$

Why useful? Assume $f : \{0, 1\}^m \rightarrow \{0, 1\}^m$
is a ONE-WAY PERMUTATION (OWP).

Then, look:

$$G(s) = f(s) \parallel h(s) \quad ; \quad s \leftarrow U_m$$

$$\approx_c f(s) \parallel b \quad ; \quad b \leftarrow \{0, 1\}^m$$

$$\equiv U_{m+1}$$

In general: The above can be made to work assuming f is a OWF (not DWP).

Moreover, every f admits hard-core but h !

Question: Can there be an $h: \{0,1\}^n \rightarrow \{0,1\}$ for every $f: \{0,1\}^n \rightarrow \{0,1\}^n$? No.

Idea: Take any h ; build f_{BAD} s.t.

h not hard-core for f_{BAD} but f_{BAD} = OWF.

It's okay if f_{BAD} relies on existing OWF $f: \{0,1\}^n \rightarrow \{0,1\}^n$